



Instituto Federal do Ceará - Embarcatech

Relatório Final do Projeto

**Sistema IoT para Medição e Análise de Consumo de
Energia Elétrica**

Autor: José Adriano Filho

Matrícula: 2025101109806

Jan/2025

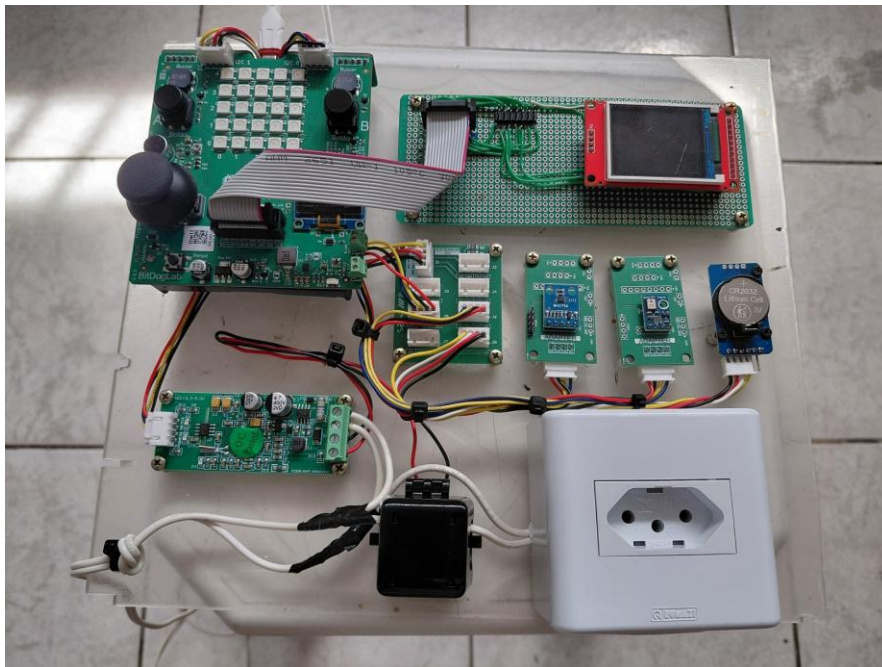
SISTEMA EMBARCADO PARA MONITORAMENTO DE ENERGIA

1 INTRODUÇÃO

A crescente evolução das tecnologias associadas à Internet das Coisas (IoT) tem possibilitado o desenvolvimento de sistemas embarcados capazes de realizar o monitoramento remoto de grandezas físicas de forma contínua, segura e eficiente. Esses sistemas vêm sendo amplamente aplicados em contextos residenciais, industriais e educacionais, permitindo a coleta de dados, o processamento local e o envio das informações para plataformas em nuvem.

No contexto dos sistemas embarcados modernos, a utilização de sistemas operacionais de tempo real (RTOS) torna-se essencial para o gerenciamento de múltiplas tarefas concorrentes, garantindo previsibilidade temporal, modularidade e escalabilidade do firmware. Associado a isso, a comunicação segura de dados é um requisito fundamental, especialmente em aplicações IoT que envolvem redes públicas.

Dessa forma, este trabalho apresenta o desenvolvimento de um sistema embarcado baseado na Raspberry Pi Pico W, utilizando o sistema operacional FreeRTOS, sensores ambientais e de energia elétrica, além de comunicação via protocolo MQTT. O sistema desenvolvido permite a aquisição, visualização local e transmissão remota dos dados, proporcionando uma solução completa de monitoramento.



2 OBJETIVOS

Desenvolver um sistema embarcado com conectividade Wi-Fi capaz de monitorar variáveis ambientais e grandezas elétricas, utilizando arquitetura multitarefa e comunicação segura com plataforma IoT.

2.2 Objetivos Específicos

- Implementar firmware em linguagem C utilizando o SDK oficial da Raspberry Pi Pico W;
- Utilizar o sistema operacional FreeRTOS para gerenciamento das tarefas;
- Integrar sensores ambientais de temperatura, umidade e luminosidade;
- Implementar a leitura de grandezas elétricas por meio de protocolo Modbus RTU;
- Exibir informações em tempo real em um display gráfico;
- Estabelecer comunicação Wi-Fi estável;
- Implementar comunicação MQTT;
- Validar o funcionamento do sistema por meio de testes práticos.

3 JUSTIFICATIVA

O monitoramento de energia elétrica é uma ferramenta essencial para a identificação de desperdícios, análise de consumo e melhoria da eficiência energética. Quando associado ao monitoramento de variáveis ambientais, possibilita uma compreensão mais abrangente do comportamento do sistema analisado.

Sob o ponto de vista acadêmico, o projeto se justifica pela integração prática de conceitos fundamentais da área de sistemas embarcados e IoT, tais como sistemas operacionais de tempo real, comunicação serial, protocolos de rede, segurança da informação e integração hardware e software.

Além disso, a solução desenvolvida apresenta potencial de aplicação em ambientes reais, podendo ser expandida para diferentes contextos, o que reforça sua relevância técnica e prática.

4 PROBLEMÁTICA

Durante o desenvolvimento do sistema, foram identificados desafios técnicos relacionados à escolha dos sensores, à comunicação segura e à integração multitarefa. Inicialmente, foi utilizado um módulo de medição de energia baseado no circuito integrado HLW8032, que apresentou dificuldades relacionadas à calibração, instabilidade nas leituras e maior complexidade de implementação.

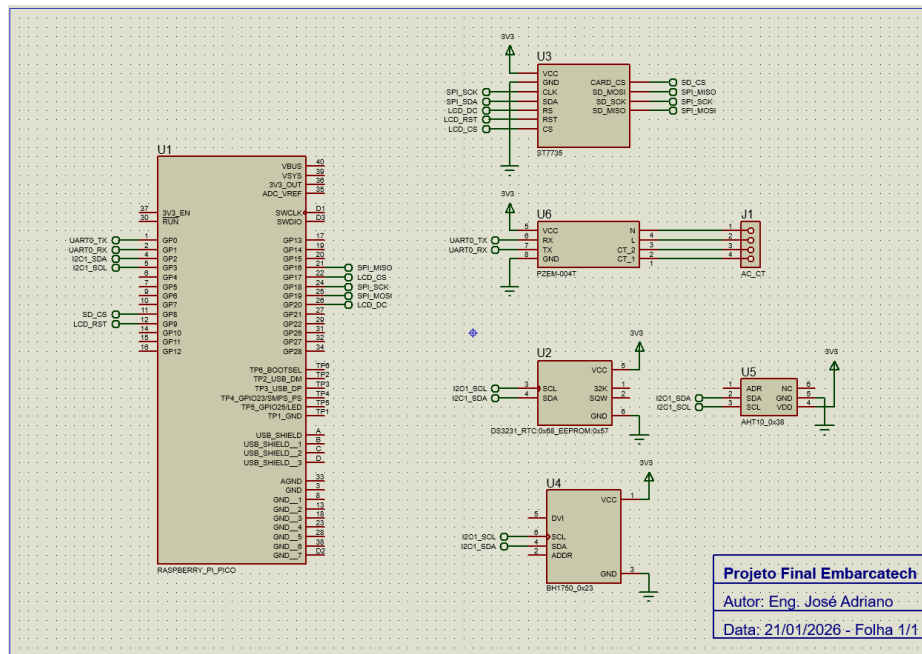
Outro desafio relevante foi a implementação de comunicação MQTT segura utilizando TLS em um microcontrolador com recursos limitados, exigindo cuidados especiais com o gerenciamento de memória, certificados digitais e configuração do stack de rede.

Como solução, optou-se pela substituição do módulo HLW8032 pelo PZEM-004T versão 4.0, que utiliza protocolo Modbus RTU e apresentou maior confiabilidade e facilidade de integração. No firmware, adotou-se uma arquitetura modular, utilizando filas (queues) do FreeRTOS para comunicação entre tarefas, garantindo maior robustez e organização do sistema.

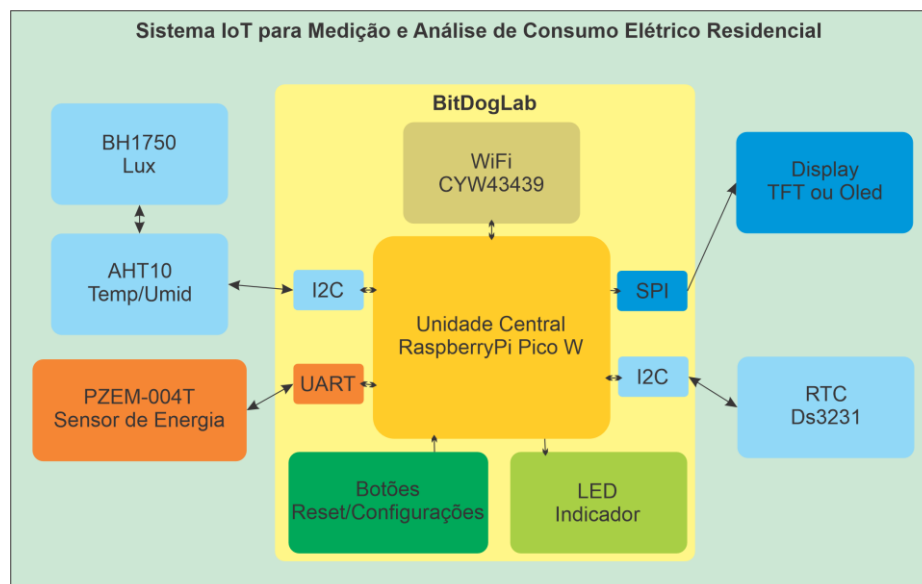
5 ARQUITETURA DO SISTEMA

5.1 Arquitetura de Hardware

O sistema de hardware é composto por uma Raspberry Pi Pico W como unidade central de processamento, sensores ambientais para medição de temperatura, umidade e luminosidade, um módulo de medição de energia elétrica, um display gráfico para interface local, além de módulos auxiliares como RTC, botões e LEDs.



A comunicação entre os sensores ambientais e o microcontrolador ocorre por meio do barramento I2C, enquanto o módulo de energia utiliza comunicação UART com protocolo Modbus RTU. O display gráfico é conectado via SPI.



PINOS DE CONTROLE PROJETO FINAL - EmbarcaTech	
Sistema IoT para Medição e Análise de Consumo Elétrico Residencial	
Descrição	GPIO
UART_TX - Pino de transmissão para o módulo sensor de energia	GPIO_0
UART_RX - Pino de recepção para o módulo sensor de energia	GPIO_1
I2C1_SDA - Pino de dados da I2C para os sensores, RTC e memória	GPIO_2
I2C1_SCL - Pino de clock da I2C para os sensores, RTC e memória	GPIO_3
SD_CS - Pino de chip select do cartão SD no LCD	GPIO_8
LCD_RST - Pino de reset do LCD ST7735	GPIO_9
SPI_MISO - Pino de Master Input Slave Output do LCD ST7735	GPIO_16
LCD_CS - Pino de chip select do LCD ST7735	GPIO_17
SPI_SCK - Pino de clock da interface SPI do LCD ST7735	GPIO_18
SPI_MOSI - Pino de Master Output Slave Input do LCD ST7735	GPIO_19
LCD_DC (RS) - Pino de controle do LCD ST7735	GPIO_20

5.2 Arquitetura de Firmware

O firmware foi desenvolvido de forma modular, organizado em camadas de drivers, serviços e aplicação. O sistema operacional FreeRTOS é responsável pelo escalonamento das tarefas, garantindo execução determinística e isolamento funcional entre os módulos.

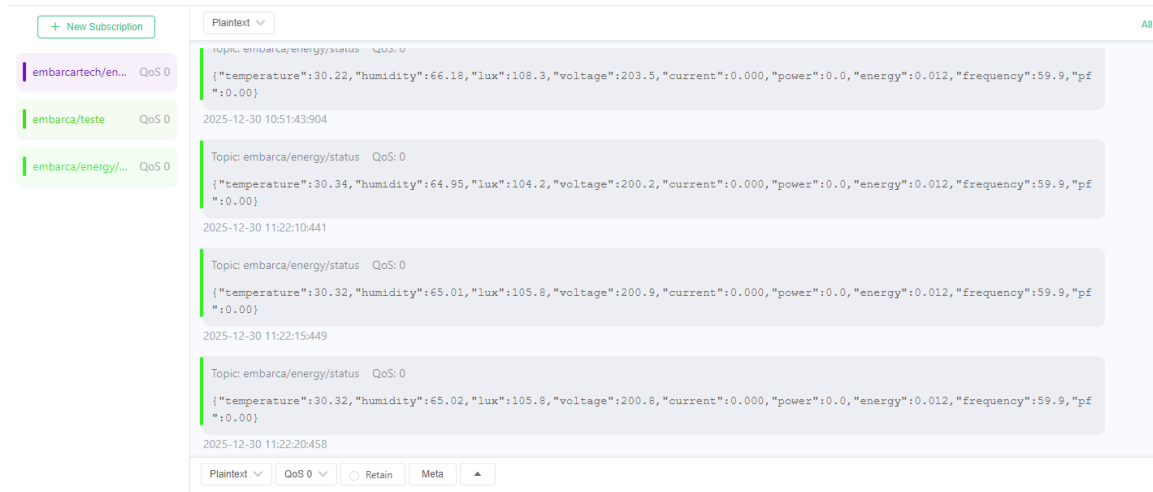
```

energy_monitoring/
├── app/
│   ├── app_init.c
│   ├── app_init.h
│   ├── app_tasks.c
│   └── app_tasks.h
├── core/
│   ├── system_state.c
│   ├── system_state.h
│   ├── time_manager.c
│   └── time_manager.h
├── drivers/
│   ├── aht10/
│   ├── bh1750/
│   ├── pzem004t/
│   ├── rtc_ds3231/
│   ├── eeprom_at24c32/
│   └── st7735/
├── services/
│   ├── wifi_manager.c
│   ├── mqtt_manager.c
│   ├── mqtt_payload.c
│   ├── mqtt_topics.h
│   └── web_server.c
├── tasks/
│   ├── task_env_sensors.c
│   ├── task_energy.c
│   ├── task_mqtt.c
│   ├── task_ui.c
│   └── task_web.c
├── include/
│   └── project_config.h
├── main.c
├── CMakeLists.txt
└── README.md

```

5.3 Arquitetura IoT

A camada IoT é responsável pela transmissão dos dados coletados para a nuvem, utilizando o protocolo MQTT. Os dados são publicados periodicamente em tópicos específicos, permitindo sua visualização em dashboards e armazenamento para análise histórica.



6 DESENVOLVIMENTO DO SISTEMA

O desenvolvimento do sistema envolveu a implementação de tarefas independentes no FreeRTOS, cada uma responsável por uma funcionalidade específica, como leitura de sensores, atualização do display e comunicação MQTT. A troca de informações entre tarefas é realizada por meio de filas, evitando acesso concorrente direto aos recursos compartilhados.

```
C energy.c > ...
1  #include <stdio.h>
2  #include "pico/stdlib.h"
3
4  #include "FreeRTOS.h"
5  #include "task.h"
6
7  /* App */
8  #include "app_init.h"
9
10 int main(void)
11 {
12     stdio_init_all();
13
14     //printf("Iniciando Monitorament de energia..\n");
15
16     app_init();
17
18     vTaskStartScheduler();
19
20     while (true) {}
21 }
```

```

40 void app_init(void)
41 {
42     /* ----- BOOT VISUAL RÁPIDO ----- */
43     ST7735_Init();
44     ST7735_FillScreen(ST7735_BLACK);
45     ST7735_SetRotation(1); /* Paisagem: 160x128 */
46     ST7735_DrawString(25, 50, "Inicializando...", Font_7x10, ST7735_WHITE, ST7735_BLACK);
47
48     /* Estado global */
49     system_state_init();
50
51     /* ----- FACTORY RESET ----- */
52     factory_reset_init();
53     factory_reset_task_start();
54
55     /* Eventos da UI */
56     ui_event_group = xEventGroupCreate();
57
58     /* Botão */
59     ui_button_init();
60
61     /* Wi-Fi */
62     wifi_manager_init();
63
64     /* MQTT */
65     mqtt_manager_init();
66
67     /* Tasks */
68     app_tasks_start();
69 }

```

```

16 void app_tasks_start(void)
17 {
18     /* Tasks de aquisição */
19     task_env_sensors_start();
20     task_energy_start();
21     time_manager_start();
22
23     /* Task UI (display) */
24     task_ui_start();
25
26     /* Task lwIP (infraestrutura) */
27     xTaskCreate(
28         vTaskLWIPPoll,
29         "lwip",
30         512,
31         NULL,
32         3,
33         NULL
34     );
35
36     /* Tasks MQTT */
37     xTaskCreate(
38         vTaskMQTTConnection,
39         "mqtt_conn",
40         1024,
41         NULL,
42         2,
43         NULL
44     );
45
46     xTaskCreate(
47         vTaskMQTTPublisher,
48         "mqtt_pub",
49         1024,
50         NULL,
51         2,
52         NULL
53     );
54
55     /* Task de payload MQTT */
56     xTaskCreate(
57         vTaskMQTTPayload,
58         "mqtt_payload",
59         1024,
60         NULL,

```


Os protocolos de comunicação utilizados incluem I2C, UART, SPI e MQTT, garantindo integração eficiente entre os diversos módulos do sistema.

```
1  #ifndef MQTT_MANAGER_H
2  #define MQTT_MANAGER_H
3
4  #include <stdbool.h>
5
6  #ifdef __cplusplus
7  extern "C" {
8  #endif
9
10 /* Tipo de mensagem MQTT */
11 typedef struct {
12     char topic[64];
13     char payload[512];
14 } mqtt_message_t;
15
16 /* API pública */
17 void mqtt_manager_init(void);
18 void mqtt_publish_async(const char *topic, const char *payload);
19
20 void vTaskMQTTConnection(void *pv);
21 void vTaskMQTTPublisher(void *pv);
22
23 bool mqtt_is_connected(void);
24
25 #ifdef __cplusplus
26 }
27 #endif
28
29 #endif /* MQTT_MANAGER_H */
30
```

7 EVIDÊNCIAS DE FUNCIONAMENTO

Como evidências do funcionamento do sistema, foram coletados registros fotográficos da montagem do hardware, capturas do display exibindo dados em tempo real, logs de comunicação MQTT comprovando a publicação dos dados e capturas de tela da plataforma IoT.

Adicionalmente, foi produzido um vídeo demonstrativo com duração entre cinco e sete minutos, apresentando o funcionamento completo do sistema, desde a inicialização até a visualização dos dados na nuvem.

8 CONCLUSÃO

O sistema embarcado desenvolvido atendeu aos objetivos propostos, demonstrando a viabilidade da integração entre hardware, firmware multitarefa e serviços IoT em uma plataforma de baixo custo. A utilização do FreeRTOS proporcionou maior organização e previsibilidade ao firmware, enquanto a comunicação MQTT garantiu a transmissão dos dados.

O projeto apresenta potencial de expansão, podendo ser adaptado para novas aplicações e integrações futuras.

9 REFERÊNCIAS

FREERTOS. FreeRTOS Reference Manual. Disponível em: <<https://www.freertos.org>>.

Acesso em: 1 jan. 2026.

MQTT. MQTT Version 3.1.1 Specification. Disponível em: <<https://mqtt.org>>. Acesso em: 1 jan. 2026.

RASPBERRY PI FOUNDATION. Raspberry Pi Pico W Documentation. Disponível em: <<https://www.raspberrypi.com>>. Acesso em: 1 jan. 2026.

10 APÊNDICES

Os apêndices incluem o código-fonte completo do projeto, disponibilizado em repositório GitHub, os esquemáticos elétricos, diagramas adicionais e arquivos de configuração utilizados durante o desenvolvimento.